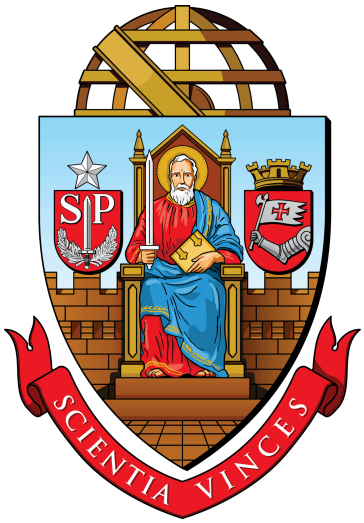




UNIVERSIDADE DE SÃO PAULO

ESCOLA POLITÉCNICA

Departamento de Engenharia de Computação e Sistemas Digitais

PCS3115 - Sistemas Digitais I

EP-01

Aproximação de cos(x)

Vinicius Castelli

NUSP: **17864509**

Professores:

Prof. Dr. Edson Satoshi Gomi

Prof. Dr. Bruno Albertini de Carvalho

São Paulo

2026

1. Introdução

Este EP implementa em Verilog um circuito combinatório que calcula $y = 1 - x^2/2$, os dois primeiros termos da série de Maclaurin de cos x. A entrada x tem 4 bits e a saída y tem 8 bits, ambas inteiras em complemento de 2. O circuito usa só descrição estrutural e dataflow, com um único projeto de somador ripple-carry reaproveitado no valor absoluto, no multiplicador e na subtração.

2. Representação numérica

Com 4 bits em complemento de 2, x vai de −8 a 7. O quadrado x^2 fica em [0, 64]. O máximo vem de $x = -8$, porque $7^2 = 49$. A metade truncada $\lfloor x^2/2 \rfloor$ fica em [0, 32] e a saída $y = 1 - \lfloor x^2/2 \rfloor$ fica em [−31, 1].

Um inteiro de 8 bits com sinal representa [−128, 127], então a saída cabe com folga. Seis bits já bastariam ([−32, 31]), mas a interface fixa 8. Internamente, x^2 é tratado como número sem sinal de 8 bits; o valor 64 é 0100_0000 e o bit 7 do produto é sempre 0.

Quando x é ímpar, $x^2/2$ não é inteiro, e foi preciso escolher um arredondamento. Escolhi o floor por dois motivos. O primeiro é numérico: $\cos x \geq 1 - x^2/2$ para todo x real. A função $f(x) = \cos x - 1 + x^2/2$ tem $f(0) = f'(0) = 0$ e $f''(x) = 1 - \cos x \geq 0$, logo é convexa com mínimo 0 em x = 0. Truncar $x^2/2$ para baixo sobe y em 0,5 nos ímpares, na direção do cos real. O segundo é de hardware: dividir um número binário por 2 e truncar é descartar o bit menos significativo, o que se faz só com fios.

3. Arquitetura

O circuito tem quatro blocos em cascata:

x[3:0] → [|x|] → a[3:0] → [a × a] → sq[7:0] → [÷2] → half[7:0] → [1 − half] → y[7:0]

Todos os blocos aritméticos usam o mesmo somador. O full adder fa é descrito em dataflow por $s = a \oplus b \oplus cin$ e $cout = a \cdot b + cin \cdot (a \oplus b)$. O add4 liga 4 instâncias de fa em ripple-carry, com o cout de cada estágio no cin do seguinte (Wakerly, seção 8.1). O add8 liga dois add4, com o cout da parte baixa no cin da parte alta. As instâncias são escritas à mão, sem generate nem parameter. No total o circuito usa 4 add4 e 1 add8, ou seja, 24 full adders, mais 16 portas AND nos produtos parciais, 4 XOR no valor absoluto e 8 inversores na subtração. O arquivo cos.v não contém nenhum operador aritmético do Verilog: só assign, ~

& | ^, concatenação, replicação, part-select e instanciação.

3.1 Valor absoluto

Como $x^2 = |x|^2$, o quadrado pode ser feito por um multiplicador sem sinal, que é o caso tratado no livro. A negação em complemento de 2 é inverter os bits e somar 1 (Wakerly, seção 2.6). O bloco calcula a = (x ⊕ {4{x[3]}}) + x[3] com um add4: o bit de sinal controla a inversão pelas XOR e entra como carry de entrada. Para $x \geq 0$ a XOR é com 0000 e o carry é 0, então a = x. Para $x < 0$ os bits são invertidos e somados a 1. Em x = −8, 1000 vira 0111 + 1 = 1000. Lido com sinal, isso seria −8 de novo, mas o bloco seguinte lê a sem sinal, e 1000 sem sinal é 8, que é o valor correto de |−8|. O cout do add4 é ignorado.

3.2 Multiplicador

O quadrado usa o método de deslocamento e soma da seção 2.8 do Wakerly, montado como multiplicador array 4×4 sem sinal. O produto parcial i é pp_i = a AND a[i], isto é, cada bit de a passa por uma AND com o bit a[i]. As somas são feitas por três add4 em sequência. Em cada etapa, o bit menos significativo da soma acumulada sai como bit final do produto, e os três bits restantes, junto com o carry, entram deslocados na etapa seguinte. O deslocamento é só ligação de fios.

Etapa	Operação	Bit de saída
0	(nenhuma)	p[0] = pp0[0]
1	{c1, s1} = pp1 + {0, pp0[3:1]}	p[1] = s1[0]
2	{c2, s2} = pp2 + {c1, s1[3:1]}	p[2] = s2[0]
3	{c3, s3} = pp3 + {c2, s2[3:1]}	p[3] = s3[0], p[7:4] = {c3, s3[3:1]}

O resultado é sq = p[7:0]. Os carries de entrada dos três add4 são 0.

3.3 Metade

A divisão por 2 com floor é half = {0, sq[7:1]}: o bit 0 é descartado e um 0 entra no bit 7. O bloco

não tem nenhuma porta.

3.4 Subtração

Falta calcular 1 − half. Em complemento de 2, inverter todos os bits de um número dá o seu negativo menos 1: ~half = −half − 1. Somando 2 dos dois lados, ~half + 2 = 1 − half. O bloco é um add8 com A = ~half, B = 0000_0010 e cin = 0. A inversão custa 8 portas NOT e a constante é ligada direto nas entradas. O cout é ignorado, porque o resultado sempre cabe em 8 bits com sinal.

4. Verificação

O testbench tb_cos.v percorre os 16 valores de x, de −8 a 7. Para cada um, aplica x, espera 10 unidades de tempo e calcula o valor esperado e = 1 − (i·i)/2 com a aritmética inteira do simulador. Como i·i ≥ 0, a divisão inteira do Verilog já é o floor. A comparação é de padrão de bits entre y e e[7:0], sem conversão de sinal. O testbench imprime uma linha CSV por caso, com x em decimal, os 8 bits de y como saíram do circuito e o valor esperado em decimal. No fim, imprime PASS se não houver divergência ou FAIL seguido do número de divergências. A coluna y_circuito da seção 5 é a leitura desses bits em complemento de 2.

O teste exaustivo é viável porque a entrada tem só 4 bits, o que dá 16 combinações, e o circuito é combinatório, sem estado interno. Os 16 casos cobrem todo o comportamento possível. Com o Icarus Verilog (iverilog -o sim cos.v tb_cos.v && vvp sim) o resultado foi PASS, com 0 divergências. A saída coincide com a tabela da seção 5.

Também conferi os sinais internos à mão para quatro entradas que exercitam casos diferentes: zero, o menor valor não nulo, o maior positivo e o −8. Os valores abaixo batem com os sinais lidos na simulação.

x	a	pp0, pp1, pp2, pp3	sq	half	y
0000 (0)	0000	0000, 0000, 0000, 0000	0000_0000 (0)	0000_0000 (0)	0000_0001 (1)
0001 (1)	0001	0001, 0000, 0000, 0000	0000_0001 (1)	0000_0000 (0)	0000_0001 (1)
0111 (7)	0111	0111, 0111, 0111, 0000	0011_0001 (49)	0001_1000 (24)	1110_1001 (−23)

x	a	pp0, pp1, pp2, pp3	sq	half	y
1000 (−8)	1000	0000, 0000, 0000, 1000	0100_0000 (64)	0010_0000 (32)	1110_0001 (−31)

Para x = 7, por exemplo, a etapa 1 soma 0111 + 0011 = 1010, a etapa 2 soma 0111 + 0101 = 1100 e a etapa 3 soma 0000 + 0110 = 0110, sem carries. Os bits de saída formam 0011_0001 = 49. Depois, ~0001_1000 = 1110_0111, e 1110_0111 + 0000_0010 = 1110_1001 = −23.

5. Comparação com cos x

A tabela reproduz o arquivo comparacao.csv, gerado a partir da saída da simulação. A coluna x é a entrada e y_circuito é a saída do circuito na simulação, $1 - \lfloor x^2/2 \rfloor$. A coluna maclaurin_exato é a série de dois termos sem arredondamento, $1 - x^2/2$, e cos_x é o cosseno real, com x em radianos.

As duas últimas colunas separam o erro em duas etapas. O erro de arredondamento, y − maclaurin, mede quanto o circuito se afasta da série exata por trabalhar só com inteiros. O erro de truncamento, maclaurin − cos x, mede quanto a série se afasta do cosseno por ter sido cortada no termo de x^2, e existiria mesmo com precisão infinita. Somando as duas colunas, o termo maclaurin se cancela e sobra o erro total y − cos x. Em x = 5, por exemplo, o circuito dá −11, a série exata dá −11,5 e o cosseno vale 0,2837. Os erros são 0,5000 e −11,7837, e o erro total é −11,2837.

x	y = cos(x) (No circuito)	MacLaurin	Cos(x) (rad)	Erro de Arredondamento	Erro de Truncamento
−8	−31,0000	−31,0000	−0,1455	0,0000	−30,8545
−7	−23,0000	−23,5000	0,7539	0,5000	−24,2539
−6	−17,0000	−17,0000	0,9602	0,0000	−17,9602
−5	−11,0000	−11,5000	0,2837	0,5000	−11,7837
−4	−7,0000	−7,0000	−0,6536	0,0000	−6,3464
−3	−3,0000	−3,5000	−0,9900	0,5000	−2,5100
−2	−1,0000	−1,0000	−0,4161	0,0000	−0,5839
−1	1,0000	0,5000	0,5403	0,5000	−0,0403

x	y = cos(x) (No circuito)	MacLaurin	Cos(x) (rad)	Erro de Arredondamento	Erro de Truncamento
0	1,0000	1,0000	1,0000	0,0000	0,0000
1	1,0000	0,5000	0,5403	0,5000	−0,0403
2	−1,0000	−1,0000	−0,4161	0,0000	−0,5839
3	−3,0000	−3,5000	−0,9900	0,5000	−2,5100
4	−7,0000	−7,0000	−0,6536	0,0000	−6,3464
5	−11,0000	−11,5000	0,2837	0,5000	−11,7837
6	−17,0000	−17,0000	0,9602	0,0000	−17,9602
7	−23,0000	−23,5000	0,7539	0,5000	−24,2539

O erro de arredondamento vem do circuito. Ele vale 0,5000 para todo x ímpar e 0 para todo x par, porque x^2 é ímpar só quando x é ímpar, e então o floor descarta exatamente 0,5. É limitado a 0,5 e foi escolhido de propósito. Como o erro de truncamento é sempre negativo, o floor reduz o erro total nos ímpares: em x = 3, o circuito fica a 2,0100 do cos, contra 2,5100 do valor exato de Maclaurin.

O erro de truncamento vem de cortar a série no termo de x^2, e não depende do hardware. O primeiro termo desprezado é $x^4/24$, que cresce rápido. Em |x| = 1 ele vale −0,0403, menor que o arredondamento de 0,5. A partir de |x| = 2 ele domina: −0,5839 em ±2, −2,5100 em ±3 e −30,8545 em x = −8, onde o circuito dá −31 e cos(−8) ≈ −0,1455. O cos fica sempre em [−1, 1], enquanto y chega a −31. Para $|x| \geq 2$ a saída não serve como aproximação de cos x, mesmo com o circuito correto.

6. Conclusão

O circuito passou nos 16 casos, e o único erro que ele introduz é o floor, de no máximo 0,5. O erro grande vem de truncar a série no segundo termo, que só acompanha cos x para $|x| \leq 1$. Cobrir a faixa [−8, 7] com precisão exigiria mais termos da série ou redução do argumento, o que está fora do escopo deste EP.

Referências Bibliográficas

Aulas do Curso.

WAKERLY, John F. Digital design: principles and practices. 5th ed. Hoboken: Pearson, 2018